can validate the parameters not only on entry to a smart contract function, but also at various stages of computation inside a function. Using `assert(winningVoteCount>=3)` will cause the function to revert if a vote count of 1 or 2 is the highest or number of voters is less than 3.

> **NOTE** The value of 3 is used here for quick testing. In a more realistic case, you could use the value for majority in the `assert()` function or some other exceptional condition that should be checked.

Listing 3.7 shows the Ballot smart contract code with these incremental improvements: another modifier, `onlyChair`, and the `assert()` function. The combination of `revert()`, `require()`, and `assert()` along with modifiers and their proper use will help you address exceptions through verification and validation, resulting in robust trust intermediation by the smart contracts. `require()` used instead of an `if` statement means that a transaction will be reverted if a condition fails. If the function call reverts, no Tx is recorded on the blockchain for this function call. It is critical to understand that `revert()` stops the Tx from happening.

Listing 3.7 With all trust rules coded (BallotV4.sol)

```
// modifiers
modifier validPhase(Phase reqPhase)                                    ❶
 { require(state == reqPhase);
   _;
 }
 modifier onlyChair()                                                  ❶
  {require(msg.sender == chairperson);
   _;
  }

 constructor (uint numProposals) public  {
     chairperson = msg.sender;
     voters[chairperson].weight = 2; // weight 2 for testing purposes
     for (uint prop = 0; prop < numProposals; prop ++)
         proposals.push(Proposal(0));
     state = Phase.Regs;
 }
```